THRESH PORTAL - ENTERPRISE SAML SSO + SUPABASE TEAMS SETUP

Complete Step-by-Step Implementation Guide

Time Required: ~4-5 hours total **Complexity:** Medium (but more powerful than OAuth) **Best For:** Enterprise team management, automatic provisioning, org-wide control

---

TABLE OF CONTENTS

1. **PHASE 1: Google Workspace Setup** (30 min)
2. **PHASE 2: Supabase Teams Configuration** (20 min)
3. **PHASE 3: SAML Implementation in Next.js** (60 min)
4. **PHASE 4: Local Testing** (20 min)
5. **PHASE 5: Vercel Deployment** (15 min)
6. **PHASE 6: Team & Role Management** (30 min)
7. **PHASE 7: Migration to Company Account** (30 min)

PREREQUISITES

Before starting, you need:

- ✅ A Google Workspace account (or trial: https://workspace.google.com/business/all/)
- ✅ Admin access to Google Workspace
- ✅ Access to Supabase account (same one from before, or new one)
- ✅ Your Next.js Thresh Portal project ready
- ✅ Vercel account

If you don't have Google Workspace yet:

1. Go to https://workspace.google.com/business/all/
2. Click **"Start your free trial"**
3. Follow setup: company name, domain (use `threshconsulting.com`), users
4. Takes ~10 minutes
5. Return here when done

WHY SAML IS BETTER THAN OAUTH

Feature	OAuth 2.0	SAML
Sync with Workspace	❌ Manual	✅ Automatic
Directory management	❌ No	✅ Yes
Multi-tenant support	❌ No	✅ Yes (Teams)
Admin controls	❌ No	✅ Workspace Admin Console
Deprovisioning	❌ Manual	✅ Automatic
Team/group management	❌ No	✅ Yes
Audit logs	❌ Basic	✅ Comprehensive
Enterprise-grade	❌ No	✅ Yes

SAML = Google Workspace syncs directly with your app

ARCHITECTURE OVERVIEW

```
Google Workspace Admin Console
        ↓
  (SAML Configuration)
        ↓
User logs in to Thresh Portal
        ↓
Redirected to Google Workspace login
        ↓
Workspace validates identity (SAML Assertion)
        ↓
Supabase receives SAML token
        ↓
Supabase Teams creates/updates user + team membership
        ↓
User logged into Thresh Portal
```

```
↓
Automatic sync of team roles & permissions
```

PHASE 1: GOOGLE WORKSPACE SETUP (30 MINUTES)

Step 1.1: Access Google Workspace Admin Console

1. Go to https://admin.google.com
2. Sign in with your Google Workspace admin account
3. You see the **Admin Console dashboard**

✅ **Result:** You're in Workspace Admin

Step 1.2: Set Up SAML Application

1. **Left sidebar → Apps → Web and mobile apps**
2. Click **"Add app" → "Add custom SAML app"**
3. **App name:** `Thresh Consulting Portal`
4. Click **CONTINUE**

You're now on the SAML details page:

Step 1.3: Download Google's SAML Metadata

On the SAML details page, you see a section: **"Google IdP information"**

1. Click **"DOWNLOAD METADATA"** button
2. A file downloads: `GoogleIDPMetadata-[yourcompany].xml`
3. **Save this file** - you need it for Supabase in Phase 2

Content of this file looks like:

```
xml
```

```
<?xml version="1.0" encoding="UTF-8"?>
<EntityDescriptor xmlns="urn:oasis:names:tc:SAML:2.0:metadata" ...>
  <IDPSSODescriptor ...>
    <SingleSignOnService Binding="urn:oasis:names:tc:SAML:2.0:bindings:HTTP-Redirect"
      Location="https://accounts.google.com/o/saml2/idp?idpid=XXXXXXX" />
  </IDPSSODescriptor>
</EntityDescriptor>
```

✅ **Result:** Google SAML metadata downloaded

Step 1.4: Configure SAML Service Provider Details

Now you need to tell Google about your app (Supabase).

In the same SAML details page, scroll to **"Service Provider Details"**

You need to fill in:

- **ACS URL** (Assertion Consumer Service URL)
- **Entity ID**

But we don't have these yet - Supabase provides them in Phase 2.

For now: Leave this page open in a tab, we'll come back to it.

Step 1.5: Configure User Access

1. Still in the SAML app page, scroll to **"User access"**
2. You see **"ON for everyone"** is greyed out
3. Click **"ON for everyone"** to enable
4. A popup: "Turn on service for everyone?"
5. Click **"TURN ON"**

All Google Workspace users can now access the app (we'll refine this in Phase 6)

✅ **Result:** SAML app configured (partially - we'll finish after Supabase)

PHASE 2: SUPABASE TEAMS CONFIGURATION (20 MINUTES)

Step 2.1: Create or Use Existing Supabase Project

If you already have a Supabase project from Phase 2 earlier: **Great, skip to 2.2**

If not:

1. Go to https://supabase.com
2. Click **"New project"**
3. **Name:** Thresh Consulting Portal
4. **Password:** Generate strong password, save it
5. **Region:** Closest to you
6. **Plan:** Free
7. Click **CREATE NEW PROJECT**
8. Wait 2-3 minutes for project to spin up

✅ **Result:** Supabase project exists

Step 2.2: Enable Supabase Teams

1. In your Supabase project, go to **Settings** (gear icon, bottom-left)
2. Click **"Teams"** in left sidebar
3. You see: **"Teams"** toggle - toggle it **ON**
4. Click **"Save"**

This enables multi-tenant team support

✅ **Result:** Teams feature enabled

Step 2.3: Configure SAML in Supabase

1. In your Supabase project, go to **Authentication**
2. Click **"Providers"** in left sidebar
3. Scroll down to **"SAML"** section
4. Click **"SAML"** to expand
5. Toggle **"Enable SAML"** to ON

Now you see a form with:

- **Metadata URL or XML**
- **Entity ID** (Supabase gives you this)
- **ACS URL** (Supabase gives you this)

Step 2.4: Upload Google SAML Metadata to Supabase

1. In the SAML section, find "**Metadata**" input
2. Click "**Upload XML**" (or paste XML)
3. Upload the `GoogleIDPMetadata-[yourcompany].xml` file you downloaded in Phase 1.3
4. Click **SAVE**

Supabase now reads your Google Workspace SAML config

Step 2.5: Get Supabase's SAML Details

After saving, Supabase shows you:

1. **Entity ID** (looks like: `urn:supabase:yourproject`)
2. **ACS URL** (looks like: `https://yourproject.supabase.co/auth/v1/sso/saml/acs`)
3. **Single Sign On URL** (for testing)

🔴 **IMPORTANT:** Copy both **Entity ID** and **ACS URL**

You need these to complete Google Workspace setup.

Step 2.6: Go Back to Google Workspace - Fill in SAML Details

1. Go back to your **Google Workspace Admin Console**
2. Navigate back to your SAML app (Apps → Web and mobile apps → Thresh Consulting Portal)
3. Scroll to "**Service Provider Details**"

Fill in:

- **ACS URL:** (paste from Supabase Step 2.5)
- **Entity ID:** (paste from Supabase Step 2.5)

- **Name ID format:** EMAIL

Click **CONTINUE**

Step 2.7: Configure User Attributes in Google Workspace

Google Workspace now shows **"Attribute mapping"**

You need to map Google Workspace user attributes to SAML attributes:

Google Workspace Attribute	SAML Attribute	App Attribute
Email address	Email	email
First name	First name	given_name
Last name	Last name	family_name
Primary email	Primary email	primary_email

How to set this up:

1. For each row, Google shows a dropdown
2. Select the corresponding Google attribute
3. It auto-populates the SAML attribute

Click FINISH when done

✅ **Result:** SAML fully configured in both Google Workspace and Supabase

---

PHASE 3: NEXT.JS IMPLEMENTATION (60 MINUTES)

Step 3.1: Install SAML Libraries

```bash
npm install @supabase/supabase-js @supabase/auth-helpers-nextjs
```

(Same as OAuth - Supabase handles SAML on their side)

Step 3.2: Create `.env.local`

In your project root, create `.env.local`:

```env
NEXT_PUBLIC_SUPABASE_URL=https://yourproject.supabase.co
NEXT_PUBLIC_SUPABASE_ANON_KEY=eyJ...
NEXT_PUBLIC_SAML_ENABLED=true
```

Values from your Supabase project Settings → API

Step 3.3: Create Supabase Client

Create file: `lib/supabase.ts`

```typescript
import { createBrowserClient } from '@supabase/auth-helpers-nextjs'

export const createClient = () =>
  createBrowserClient(
    process.env.NEXT_PUBLIC_SUPABASE_URL!,
    process.env.NEXT_PUBLIC_SUPABASE_ANON_KEY!
  )
```

Step 3.4: Create Login Page with SAML Button

Create file: `app/login/page.tsx`

```typescript
```

```
'use client'

import { createClient } from '@/lib/supabase'
import { useRouter } from 'next/navigation'
import { useState } from 'react'

export default function LoginPage() {
  const router = useRouter()
  const [loading, setLoading] = useState(false)
  const [error, setError] = useState<string | null>(null)

  const handleSAMLSignIn = async () => {
    setLoading(true)
    setError(null)

    try {
      const supabase = createClient()

      // Get SAML URL from Supabase
      const { data, error: samlError } = await supabase.auth.signInWithSSO({
        domain: 'threshconsulting.com', // Your Google Workspace domain
        options: {
          redirectTo: `${location.origin}/auth/callback`,
        },
      })

      if (samlError || !data?.url) {
        setError('Failed to initiate SAML login: ' + (samlError?.message || 'Unknown error'))
        console.error('SAML error:', samlError)
        return
      }

      // Redirect to Google Workspace login
      window.location.href = data.url
    } catch (err) {
      setError('An unexpected error occurred')
      console.error('Error:', err)
    } finally {
      setLoading(false)
    }
  }

  return (
```

```
    <div className="flex items-center justify-center min-h-screen bg-gradient-to-b from-[#3a36c9] to-[#3a36c9]">
      <div className="bg-white rounded-lg shadow-lg p-8 w-full max-w-md">
        <div className="text-center mb-8">
          <h1 className="text-3xl font-bold text-gray-900 mb-2">
            Thresh Portal
          </h1>
          <p className="text-gray-600">
            Enterprise SSO with Google Workspace
          </p>
        </div>

        {error && (
          <div className="mb-4 p-3 bg-red-50 border border-red-200 rounded text-red-700 text-sm">
            {error}
          </div>
        )}

        <button
          onClick={handleSAMLSignIn}
          disabled={loading}
          className="w-full bg-[#3a36c9] text-white py-3 px-4 rounded-lg font-medium hover:bg-[#2d2a9f] disabled:opacity-5
        >
          {loading ? 'Connecting to Workspace...' : 'Sign in with Google Workspace'}
        </button>

        <div className="mt-6 pt-6 border-t border-gray-200">
          <p className="text-xs text-gray-500 text-center">
            🔒 Enterprise SSO enabled
          </p>
          <p className="text-xs text-gray-500 text-center mt-2">
            Your identity is managed by Google Workspace
          </p>
        </div>
      </div>
    </div>
  )
}
```

Step 3.5: Create SAML Callback Handler

Create file: `app/auth/callback/route.ts`

```typescript
import { createClient } from '@/lib/supabase'
import { NextRequest, NextResponse } from 'next/server'

export async function GET(request: NextRequest) {
  const { searchParams } = new URL(request.url)
  const code = searchParams.get('code')
  const error = searchParams.get('error')

  // Handle SAML errors
  if (error) {
    return NextResponse.redirect(
      new URL(`/login?error=${encodeURIComponent(error)}`, request.url)
    )
  }

  // Exchange SAML code for session
  if (code) {
    const supabase = createClient()
    const { error: exchangeError } = await supabase.auth.exchangeCodeForSession(code)

    if (exchangeError) {
      console.error('SAML exchange error:', exchangeError)
      return NextResponse.redirect(
        new URL(`/login?error=saml_exchange_failed`, request.url)
      )
    }
  }

  // SAML successful - redirect to dashboard
  return NextResponse.redirect(new URL('/dashboard', request.url))
}
```

Step 3.6: Create Middleware for Route Protection

Create file: `middleware.ts` (root level)

```typescript
```

```
import { type NextRequest, NextResponse } from 'next/server'
import { createClient } from '@/lib/supabase'

export async function middleware(request: NextRequest) {
  const supabase = createClient()

  try {
    const { data: { session } } = await supabase.auth.getSession()

    // Allow login page without auth
    if (request.nextUrl.pathname === '/login' ||
        request.nextUrl.pathname === '/auth/callback') {
      return NextResponse.next()
    }

    // Require authentication
    if (!session) {
      return NextResponse.redirect(new URL('/login', request.url))
    }

    // SAML verified user - check if they're in your team
    const userEmail = session.user.email || ''

    // Verify they're from @threshconsulting.com (belt-and-suspenders security)
    if (!userEmail.endsWith('@threshconsulting.com')) {
      return NextResponse.redirect(
        new URL(`/login?error=unauthorized_domain`, request.url)
      )
    }

    // User is authenticated and authorized
    return NextResponse.next()
  } catch (err) {
    console.error('Middleware error:', err)
    return NextResponse.redirect(new URL('/login', request.url))
  }
}

export const config = {
  matcher: ['/((?!api|_next/static|_next/image|favicon.ico).*)'],
}
```

Step 3.7: Create User Profile Table with Teams

Create file: `app/api/setup-tables/route.ts`

This endpoint sets up your database schema for SAML + Teams:

```typescript
```

```
import { createClient } from '@/lib/supabase'
import { NextRequest, NextResponse } from 'next/server'

export async function POST(request: NextRequest) {
  const supabase = createClient()

  try {
    // Create profiles table
    await supabase.rpc('exec_sql', {
      sql: `
        CREATE TABLE IF NOT EXISTS profiles (
          id UUID REFERENCES auth.users(id) PRIMARY KEY,
          email TEXT NOT NULL UNIQUE,
          full_name TEXT,
          avatar_url TEXT,
          role TEXT DEFAULT 'member', -- member, admin, owner
          team_id UUID,
          saml_verified BOOLEAN DEFAULT TRUE,
          created_at TIMESTAMP DEFAULT NOW(),
          updated_at TIMESTAMP DEFAULT NOW()
        );

        -- Enable RLS
        ALTER TABLE profiles ENABLE ROW LEVEL SECURITY;

        -- Policies
        CREATE POLICY "Users can read own profile"
          ON profiles FOR SELECT
          USING (auth.uid() = id);

        CREATE POLICY "Users can update own profile"
          ON profiles FOR UPDATE
          USING (auth.uid() = id);

        CREATE POLICY "Admins can read team profiles"
          ON profiles FOR SELECT
          USING (
            EXISTS (
              SELECT 1 FROM profiles p
              WHERE p.id = auth.uid() AND p.role = 'admin'
            )
          );
```

```
    -- Create teams table
    CREATE TABLE IF NOT EXISTS teams (
      id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
      name TEXT NOT NULL,
      slug TEXT UNIQUE NOT NULL,
      description TEXT,
      owner_id UUID REFERENCES auth.users(id),
      created_at TIMESTAMP DEFAULT NOW(),
      updated_at TIMESTAMP DEFAULT NOW()
    );

    ALTER TABLE teams ENABLE ROW LEVEL SECURITY;

    CREATE POLICY "Anyone can read teams"
      ON teams FOR SELECT
      USING (true);

    -- Create team members table
    CREATE TABLE IF NOT EXISTS team_members (
      id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
      team_id UUID REFERENCES teams(id) ON DELETE CASCADE,
      user_id UUID REFERENCES auth.users(id) ON DELETE CASCADE,
      role TEXT DEFAULT 'member', -- owner, admin, member
      created_at TIMESTAMP DEFAULT NOW(),
      UNIQUE(team_id, user_id)
    );

    ALTER TABLE team_members ENABLE ROW LEVEL SECURITY;

    CREATE POLICY "Team members can read own teams"
      ON team_members FOR SELECT
      USING (
        user_id = auth.uid() OR
        EXISTS (
          SELECT 1 FROM team_members tm
          WHERE tm.user_id = auth.uid() AND tm.team_id = team_members.team_id
        )
      );
  `
  })

  return NextResponse.json({ success: true, message: 'Tables created' })
} catch (error) {
  console.error('Setup error:', error)
```

```
    return NextResponse.json({ error: String(error) }, { status: 500 })
  }
}
```

Or use SQL Editor in Supabase directly:

Go to **SQL Editor → New Query** and run:

```sql
```

```sql
-- Profiles table
CREATE TABLE IF NOT EXISTS profiles (
  id UUID REFERENCES auth.users(id) PRIMARY KEY,
  email TEXT NOT NULL UNIQUE,
  full_name TEXT,
  avatar_url TEXT,
  role TEXT DEFAULT 'member',
  team_id UUID,
  saml_verified BOOLEAN DEFAULT TRUE,
  created_at TIMESTAMP DEFAULT NOW(),
  updated_at TIMESTAMP DEFAULT NOW()
);

ALTER TABLE profiles ENABLE ROW LEVEL SECURITY;

CREATE POLICY "Users can read own profile" ON profiles FOR SELECT USING (auth.uid() = id);
CREATE POLICY "Users can update own profile" ON profiles FOR UPDATE USING (auth.uid() = id);

-- Teams table
CREATE TABLE IF NOT EXISTS teams (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  name TEXT NOT NULL,
  slug TEXT UNIQUE NOT NULL,
  description TEXT,
  owner_id UUID REFERENCES auth.users(id),
  created_at TIMESTAMP DEFAULT NOW(),
  updated_at TIMESTAMP DEFAULT NOW()
);

ALTER TABLE teams ENABLE ROW LEVEL SECURITY;

CREATE POLICY "Anyone can read teams" ON teams FOR SELECT USING (true);

-- Team members table
CREATE TABLE IF NOT EXISTS team_members (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  team_id UUID REFERENCES teams(id) ON DELETE CASCADE,
  user_id UUID REFERENCES auth.users(id) ON DELETE CASCADE,
  role TEXT DEFAULT 'member',
  created_at TIMESTAMP DEFAULT NOW(),
  UNIQUE(team_id, user_id)
);
```

```sql
ALTER TABLE team_members ENABLE ROW LEVEL SECURITY;

CREATE POLICY "Team members can read teams" ON team_members FOR SELECT
USING (user_id = auth.uid());
```

Step 3.8: Update Navigation Component

Update file: `components/Navigation.tsx`

```typescript
```

```
'use client'

import { createClient } from '@/lib/supabase'
import { useRouter } from 'next/navigation'
import { useEffect, useState } from 'react'
import Link from 'next/link'

interface User {
  email: string
  user_metadata?: {
    full_name?: string
    picture?: string
  }
}

interface UserProfile {
  role: string
  full_name: string
}

export default function Navigation() {
  const router = useRouter()
  const [user, setUser] = useState<User | null>(null)
  const [profile, setProfile] = useState<UserProfile | null>(null)
  const [loading, setLoading] = useState(true)

  useEffect(() => {
    const getUserData = async () => {
      const supabase = createClient()

      // Get session
      const { data: { session } } = await supabase.auth.getSession()

      if (session?.user) {
        setUser(session.user as any)

        // Get user profile
        const { data: profileData } = await supabase
          .from('profiles')
          .select('role, full_name')
          .eq('id', session.user.id)
          .single()
```

```
      if (profileData) {
        setProfile(profileData)
      }
    }

    setLoading(false)
  }

  getUserData()
}, [])

const handleLogout = async () => {
  const supabase = createClient()
  await supabase.auth.signOut()
  router.push('/login')
}

if (loading) {
  return <nav className="bg-white shadow"><div className="h-16"></div></nav>
}

return (
  <nav className="bg-white shadow">
    <div className="max-w-7xl mx-auto px-4 sm:px-6 lg:px-8">
      <div className="flex justify-between items-center h-16">
        <Link href="/dashboard" className="text-2xl font-bold text-[#3a36c9]">
          Thresh Portal
        </Link>

        <div className="flex gap-6">
          <Link href="/dashboard" className="text-gray-700 hover:text-[#3a36c9]">
            Dashboard
          </Link>
          <Link href="/projects" className="text-gray-700 hover:text-[#3a36c9]">
            Projects
          </Link>
          <Link href="/team" className="text-gray-700 hover:text-[#3a36c9]">
            Team
          </Link>
          <Link href="/sops" className="text-gray-700 hover:text-[#3a36c9]">
            SOPs
          </Link>
          {profile?.role === 'admin' && (
            <Link href="/admin" className="text-gray-700 hover:text-[#3a36c9]">
```

```
            Admin
          </Link>
        )}
      </div>

      <div className="flex items-center gap-4">
        {user && (
          <>
            <div className="text-sm text-gray-700">
              <p className="font-medium">
                {profile?.full_name || user.email.split('@')[0]}
              </p>
              <p className="text-xs text-gray-500">
                {profile?.role || 'member'} • SAML verified
              </p>
            </div>
            <button
              onClick={handleLogout}
              className="px-4 py-2 bg-red-500 text-white rounded hover:bg-red-600 text-sm font-medium"
            >
              Sign Out
            </button>
          </>
        )}
      </div>
    </div>
  </div>
</nav>
)
}
```

PHASE 4: LOCAL TESTING (20 MINUTES)

Step 4.1: Start Dev Server

```bash
npm run dev
```

Step 4.2: Test SAML Login Flow

⚠️ **IMPORTANT:** SAML doesn't work well on localhost for testing (Google requires HTTPS for SAML)

Option A: Test with ngrok (recommended)

1. Install ngrok: https://ngrok.com/download
2. Run: `ngrok http 3000`
3. You get a URL like: `https://abc123.ngrok.io`
4. Update Google Workspace SAML config:
   - Set ACS URL to: `https://abc123.ngrok.io/auth/callback`
   - Set Entity ID to: `https://abc123.ngrok.io`
5. Visit: `https://abc123.ngrok.io/login`
6. Click "Sign in with Google Workspace"
7. You're redirected to Google login
8. Sign in with your @threshconsulting.com account
9. Should redirect to dashboard

Option B: Skip local testing, deploy to Vercel first

(Vercel has HTTPS automatically, so SAML works immediately)

Step 4.3: Verify in Supabase

1. Go to your Supabase project → **Authentication** → **Users**
2. You should see your user listed
3. Click on the user to view details
4. You should see:
   - Email: your@threshconsulting.com
   - Provider: `saml`
   - SAML verified: `true`

✅ **If this works: SAML integration is successful!**

PHASE 5: VERCEL DEPLOYMENT (15 MINUTES)

Step 5.1: Push to GitHub

```bash
git add .
git commit -m "Add SAML SSO with Google Workspace and Supabase Teams"
git push origin main
```

Step 5.2: Deploy to Vercel

1. Go to https://vercel.com
2. Click **"Add New..."** → **"Project"**
3. Select your Thresh Portal repo
4. Click **"Import"**

Step 5.3: Add Environment Variables

On the "Configure Project" page:

```env
NEXT_PUBLIC_SUPABASE_URL=https://yourproject.supabase.co
NEXT_PUBLIC_SUPABASE_ANON_KEY=eyJ...
NEXT_PUBLIC_SAML_ENABLED=true
```

Click **"Deploy"**

Wait 2-3 minutes...

You get: `Your site is live at: https://[name]-thresh-portal.vercel.app`

Step 5.4: Update Google Workspace SAML Config

1. Go to **Google Workspace Admin Console**
2. Find your SAML app
3. Update **Service Provider Details:**
   - **ACS URL:** `https://[name]-thresh-portal.vercel.app/auth/callback`

- **Entity ID:** Keep as is (or update to match)

4. Click **SAVE**

Step 5.5: Update Supabase SAML Config (if needed)

1. Go to Supabase → **Authentication** → **Providers** → **SAML**
2. Update ACS URL if it changed
3. Click **SAVE**

Step 5.6: Test Live Site

1. Open: `https://[name]-thresh-portal.vercel.app/login`
2. Click "Sign in with Google Workspace"
3. Sign in with your @threshconsulting.com account
4. Should redirect to dashboard

🎉 **Live SAML SSO is working!**

PHASE 6: TEAM & ROLE MANAGEMENT (30 MINUTES)

Step 6.1: Create Initial Team in Supabase

In Supabase **SQL Editor**, run:

```sql
```

```sql
-- Create default team
INSERT INTO teams (name, slug, description, owner_id)
VALUES (
  'Thresh Consulting',
  'thresh-consulting',
  'Main company team',
  (SELECT id FROM auth.users LIMIT 1) -- Replace with your user ID
);

-- Add yourself to team with admin role
INSERT INTO team_members (team_id, user_id, role)
VALUES (
  (SELECT id FROM teams WHERE slug = 'thresh-consulting'),
  (SELECT id FROM auth.users LIMIT 1),
  'admin'
);

-- Update your profile
UPDATE profiles
SET role = 'admin', team_id = (SELECT id FROM teams WHERE slug = 'thresh-consulting')
WHERE id = (SELECT id FROM auth.users LIMIT 1);
```

Step 6.2: Auto-Provision New Users

Every time a new Google Workspace user logs in via SAML, they should be added to your team automatically.

Create file: `app/api/auth/saml-provision/route.ts`

```typescript
```

```
import { createClient } from '@/lib/supabase'
import { NextRequest, NextResponse } from 'next/server'

export async function POST(request: NextRequest) {
  const supabase = createClient()

  try {
    const { data: { session } } = await supabase.auth.getSession()

    if (!session?.user) {
      return NextResponse.json({ error: 'Not authenticated' }, { status: 401 })
    }

    const user = session.user
    const email = user.email || ''

    // Verify it's a company email
    if (!email.endsWith('@threshconsulting.com')) {
      return NextResponse.json({ error: 'Invalid domain' }, { status: 403 })
    }

    // Create or update profile
    const { error: profileError } = await supabase
      .from('profiles')
      .upsert({
        id: user.id,
        email: email,
        full_name: user.user_metadata?.full_name || email.split('@')[0],
        avatar_url: user.user_metadata?.picture,
        saml_verified: true,
      }, { onConflict: 'id' })

    if (profileError) throw profileError

    // Get default team
    const { data: team, error: teamError } = await supabase
      .from('teams')
      .select('id')
      .eq('slug', 'thresh-consulting')
      .single()

    if (teamError || !team) {
      console.error('Team not found')
```

```
    return NextResponse.json({ error: 'Team configuration error' }, { status: 500 })
  }

  // Add user to team if not already a member
  const { error: memberError } = await supabase
    .from('team_members')
    .insert({
      team_id: team.id,
      user_id: user.id,
      role: 'member',
    })
    .select()

  // Ignore duplicate key errors
  if (memberError && !memberError.message.includes('duplicate')) {
    throw memberError
  }

  return NextResponse.json({ success: true })
 } catch (error) {
  console.error('Provisioning error:', error)
  return NextResponse.json({ error: String(error) }, { status: 500 })
 }
}
```

Add this to your callback route (`app/auth/callback/route.ts`):

```typescript
// After successful SAML exchange:
await fetch(`${location.origin}/api/auth/saml-provision`, {
  method: 'POST',
})
```

Step 6.3: Create Admin Dashboard

Create file: `app/admin/page.tsx`

```typescript
```

```
'use client'

import { createClient } from '@/lib/supabase'
import { useEffect, useState } from 'react'

interface TeamMember {
  id: string
  email: string
  full_name: string
  role: string
  created_at: string
}

export default function AdminPage() {
  const [members, setMembers] = useState<TeamMember[]>([])
  const [loading, setLoading] = useState(true)
  const [error, setError] = useState<string | null>(null)

  useEffect(() => {
    const fetchMembers = async () => {
      const supabase = createClient()

      const { data: { session } } = await supabase.auth.getSession()

      if (!session) return

      // Get team members
      const { data, error: queryError } = await supabase
        .from('team_members')
        .select(`
          id,
          role,
          created_at,
          user_id,
          profiles:user_id (
            email,
            full_name
          )
        `)
        .eq('team_id', (await supabase
          .from('teams')
          .select('id')
          .eq('slug', 'thresh-consulting')
```

```
        .single()).data?.id)

    if (queryError) {
      setError(queryError.message)
    } else {
      // Transform data
      setMembers(data?.map((m: any) => ({
        id: m.id,
        email: m.profiles?.email,
        full_name: m.profiles?.full_name,
        role: m.role,
        created_at: m.created_at,
      })) || [])
    }

    setLoading(false)
  }

  fetchMembers()
}, [])

const handleRoleChange = async (memberId: string, newRole: string) => {
  const supabase = createClient()

  const { error } = await supabase
    .from('team_members')
    .update({ role: newRole })
    .eq('id', memberId)

  if (error) {
    setError(error.message)
  } else {
    // Refresh members list
    setMembers(members.map(m =>
      m.id === memberId ? { ...m, role: newRole } : m
    ))
  }
}

if (loading) return <div className="p-8">Loading...</div>

return (
  <div className="max-w-7xl mx-auto px-4 sm:px-6 lg:px-8 py-8">
    <h1 className="text-4xl font-bold mb-8">Admin Panel</h1>
```

```
{error && (
  <div className="mb-4 p-3 bg-red-50 border border-red-200 rounded text-red-700">
    {error}
  </div>
)}

<div className="bg-white rounded-lg shadow">
  <table className="w-full">
    <thead className="border-b bg-gray-50">
      <tr>
        <th className="px-6 py-3 text-left text-sm font-medium text-gray-900">
          Name
        </th>
        <th className="px-6 py-3 text-left text-sm font-medium text-gray-900">
          Email
        </th>
        <th className="px-6 py-3 text-left text-sm font-medium text-gray-900">
          Role
        </th>
        <th className="px-6 py-3 text-left text-sm font-medium text-gray-900">
          Joined
        </th>
      </tr>
    </thead>
    <tbody>
      {members.map(member => (
        <tr key={member.id} className="border-b hover:bg-gray-50">
          <td className="px-6 py-4 text-sm text-gray-900">
            {member.full_name}
          </td>
          <td className="px-6 py-4 text-sm text-gray-600">
            {member.email}
          </td>
          <td className="px-6 py-4 text-sm">
            <select
              value={member.role}
              onChange={(e) =>
                handleRoleChange(member.id, e.target.value)
              }
              className="px-2 py-1 border rounded text-sm"
            >
              <option value="member">Member</option>
              <option value="admin">Admin</option>
```

```
            <option value="owner">Owner</option>
          </select>
          </td>
          <td className="px-6 py-4 text-sm text-gray-600">
           {new Date(member.created_at).toLocaleDateString()}
          </td>
         </tr>
        ))}
       </tbody>
      </table>
     </div>

     <div className="mt-8 p-4 bg-blue-50 border border-blue-200 rounded">
      <h3 className="font-semibold text-blue-900 mb-2">How it works:</h3>
      <ul className="text-sm text-blue-800 space-y-1">
       <li>✅ Any @threshconsulting.com user can sign in via Google Workspace</li>
       <li>✅ They're automatically added to the "Thresh Consulting" team</li>
       <li>✅ Admins can manage roles from this dashboard</li>
       <li>✅ Removing someone from Workspace = automatic access revocation</li>
      </ul>
     </div>
    </div>
  )
}
```

PHASE 7: MIGRATION TO COMPANY ACCOUNT (30 MINUTES)

Step 7.1: Transfer Google Workspace

1. Go to **Google Workspace Admin Console**
2. Check if you're the primary admin
3. If not, have the primary admin add you as super admin:
   - **Users and accounts** → (your email) → **Admin roles**
   - Assign "Super Admin"
4. Now you own the SAML configuration

Step 7.2: Transfer Supabase to Company

1. Supabase project → **Settings** → **Organization**
2. Look for **"Transfer project"** button
3. Enter: tech@threshconsulting.com
4. That person receives invitation to accept ownership

Step 7.3: Transfer Vercel to Company

1. Vercel project → **Settings**
2. Go to **General** tab
3. Scroll to **Ownership**
4. Click **"Change owner"**
5. Enter: tech@threshconsulting.com
6. They receive invitation

Step 7.4: Transfer GitHub Repo (Optional)

1. GitHub repo → **Settings**
2. Scroll to **Danger zone**
3. Click **Transfer**
4. Enter company organization
5. Confirm

KEY ADVANTAGES OF SAML VS OAUTH

Feature	OAuth	SAML
User added to Workspace	Must manually invite to app	✅ Auto-provisioned
User removed from Workspace	Still has access ❌	✅ Immediate revocation
Admin controls	N/A	✅ Workspace Admin Console
Team management	Manual	✅ Automatic

Feature	OAuth	SAML
Audit logging	Basic	✅ Comprehensive
Multi-tenant	No	✅ Yes (Supabase Teams)
Enterprise grade	No	✅ Yes

TROUBLESHOOTING

Problem	Solution
"SAML assertion failure"	Check ACS URL and Entity ID match in Google + Supabase
"Invalid domain"	User email doesn't end with @threshconsulting.com
Redirect loop	Check cookies and session storage enabled
"Unauthorized user"	User not in Google Workspace or disabled
Localhost not working	SAML requires HTTPS - use ngrok or deploy to Vercel
New users can't log in	Check auto-provisioning endpoint is working

---

AFTER DEPLOYMENT CHECKLIST

- ✅ Test login with test @threshconsulting.com account
- ✅ Verify user appears in Supabase Users list
- ✅ Verify user appears in team members table
- ✅ Test logout
- ✅ Test new user login (auto-provisioning)
- ✅ Test removing user from Workspace → they lose access
- ✅ Test role changes in admin panel
- ✅ Set up Google Workspace audit logging

FINAL ARCHITECTURE

```
Google Workspace (Source of Truth)
  ↓
SAML Assertion
  ↓
Supabase Auth
  ↓
Supabase Teams (Auto-sync)
  ↓
Your Next.js App
  ↓
Role-based Access Control
```

Result: Enterprise-grade SSO with automatic team management!